Express Yourself

- Use OpenOffice Writer to create a new document
- Save the file as LastFirst_ic15
  - Replace LastFirst with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 15"
- **Exercise 1**: Open Eclipse and close all open projects
  - Create a new **ic15** project
  - Import **Word.java** and **SyllableCounter.java** from the Q: drive using Import from File System
  - Snap a picture and paste it into your document

Using a Debugger

- Modern IDEs (like Eclipse) contain programs called **debuggers**, that help you locate bugs by letting you interactively follow the execution of your programs
  - You can stop and restart your program and see the contents of variables when your program is temporarily stopped
  - Using a debugger is not cost free—it takes time to set up and carry out an effective debugging session
- We'll follow sample debugging session on page 267

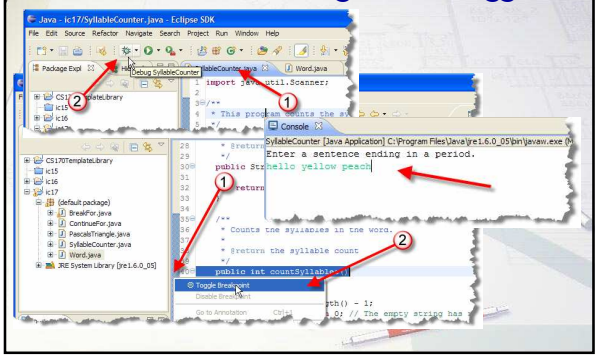
Three Useful Commands

- Larger programs are harder to debug without using a debugger, so learn how to use one is time well spent.
- Three important commands:
  - **Run** to a particular location (called a **breakpoint**)
  - **Step** to next line (step into, over)
  - **Inspect** the contents of a variable
- Exact names vary from debugger to debugger, but all support them

Run To a Particular Line

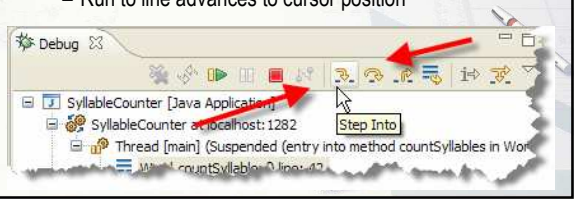
- Program is supposed to print syllables in word
- **Exercise 2**: Run **SyllabusCounter** and enter "hello yellow peach" as the input. Shoot a screen-shot.
- Let's run the program line by line
  - To do this, we need to set a breakpoint in the code
  - When the program encounters this line it will go into suspended animation, and we can examine it
- **Exercise 3**: Go to line 40, (header of countSyllables), and right-click to the left of the line number. Choose **Toggle Breakpoint** from menu. Snap a screen-shot.

Starting the Debugger



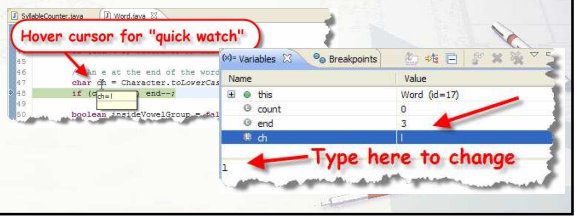
Step Over and Into

- **Exercise 4**: Shoot a screen-shot of debug display
- "Step over" and "Step into" commands advance
  - The "step over" skips over function calls
  - "Step into" advances into function calls
  - Run to line advances to cursor position

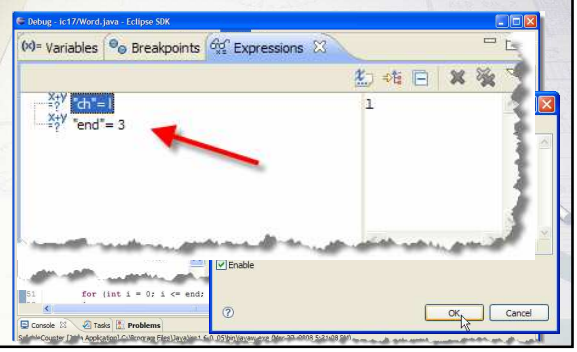


Inspecting Variables

- **Exercise 5**: put cursor on line 48, run to line. Snap.
- With program stopped, you can values of variables.
  - Eclipse shows the values of all variables and allows you to change them as well.



Watching Variables

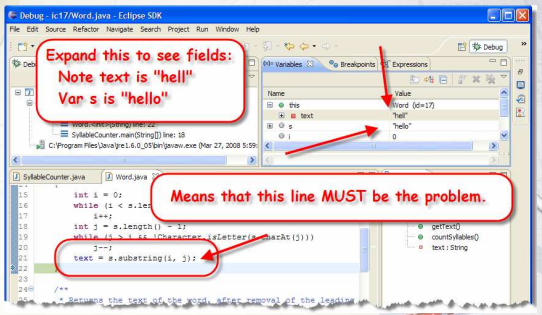


Restarting

- Note that end is 3, so we get "hell", not "hello". Can't go back in time, so Terminate session like this:

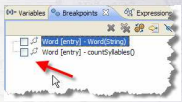
- **Exercise 6**: Put break-point on the Word constructor that creates the word passed to countSyllables
  - Add watches for variables i and j
  - Run to line 21, where text is set
  - Single-step and shoot a screenshot of variables

Tracking Down the Problem



Try Again

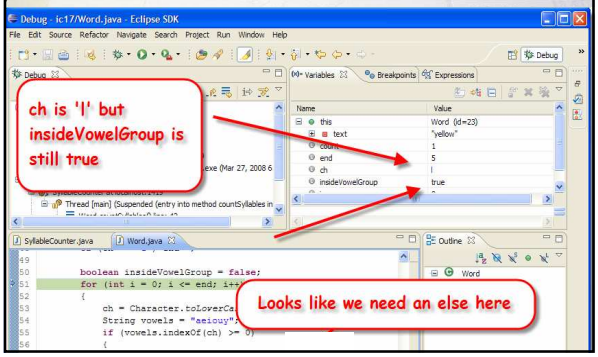
- **Exercise 7**: Fix the substring, turn off the breakpoints and run the program again. Snap a pic.



- Still a problem. Let's look closer.
  - Re-enable the breakpoint for countSyllables
  - This time, just enter "hello"
  - Single-step through countSyllables

Exercise 8: Fix, run and snap a picture.

Checking Vowels



First, You Break It

- Before you can debug your program, you must be able to reliably reproduce the error
  - What numbers did you enter?
  - Where did you click the mouse?
  - Run the same program again
  - Type in exactly the same input.
- Don't try to fix it until you can break it at will

Next, Isolate and Understand

- Use Divide and Conquer to **locate** and **isolate**
  - Step over functions, don't step into them
  - Locate the last procedure before failure
  - Step into **that** procedure, and repeat
- To **understand** the bug, you have to know what your program **should** do
  - Before inspection, know expected values
  - If value is correct, look elsewhere
  - Double-check computations

Debugging Tips

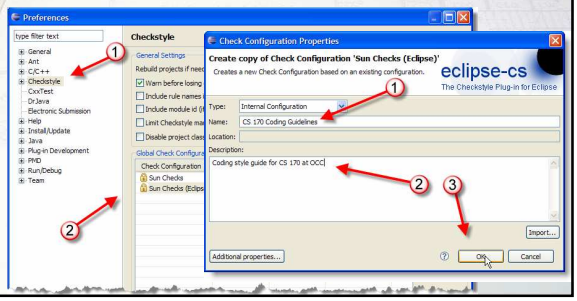
- Look for off by one errors.
- Look for computation errors
  - Is the formula typed in correctly?
- Look at "big picture" when numbers large or complex
- Avoid quick fixes - they tend to create problems elsewhere. Possibly the program logic is incorrect and the program needs to be redesigned.

A Little More Eclipse

- Configuring the **Checkstyle** Plug-in
  - Activating Checkstyle for your project
  - Creating your own rule set
  - Modifying individual rules

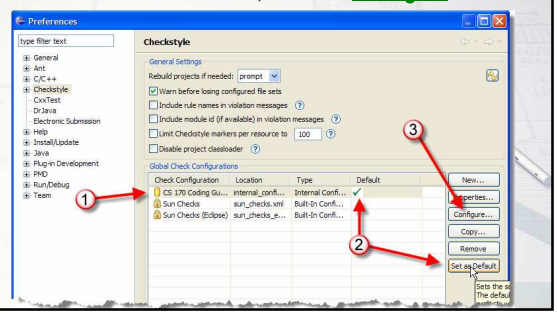
CS 170 Checkstyle

- Need to add a custom configuration for CS 170



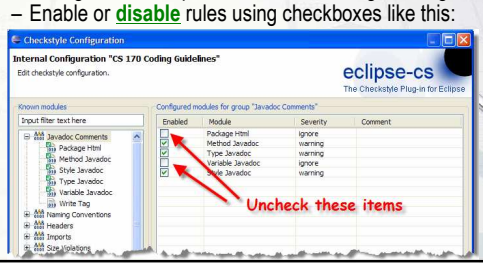
Configuring Checkstyle

- Make CS 170 the default, and click **Configure**



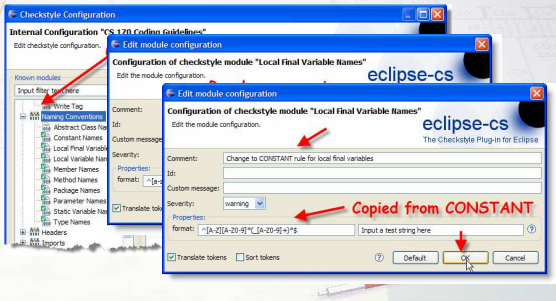
Javadoc Configuration

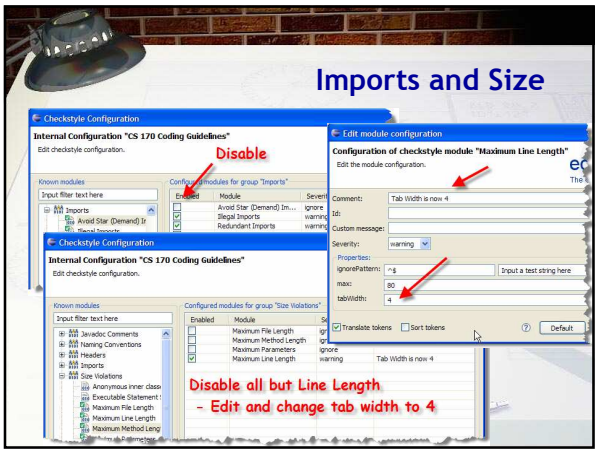
- When you configure you can do **three** things
  - Add or remove rule-sets using buttons at bottom
  - Configure how a specific rule works using a dialog
  - Enable or **disable** rules using checkboxes like this:

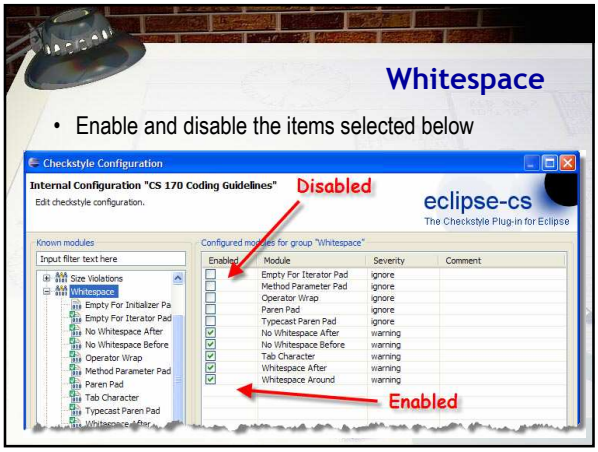


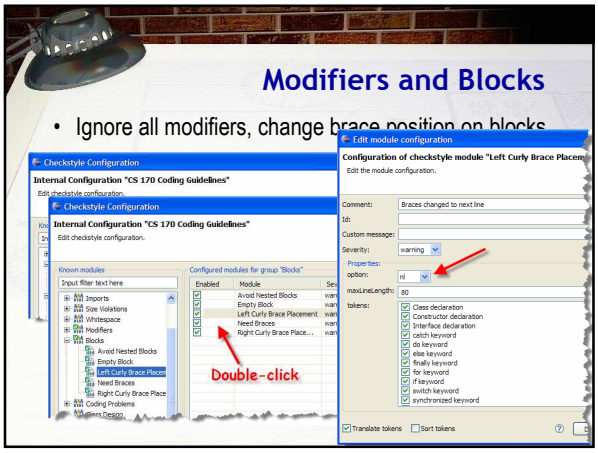
Naming Rules

- Let's change the naming rules for local constants



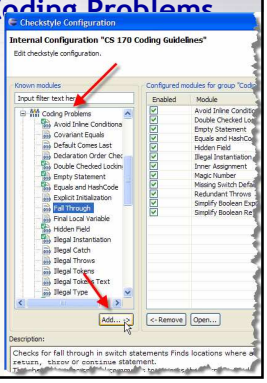

Imports and Size
Checkstyle Configuration
Internal Configuration "CS 170 Coding Guidelines"
Edit checkstyle configuration.
Disable
Edit module configuration
Configuration of checkstyle module "Maximum Line Length"
Edit the module configuration.
Comment:
Tab Width is now 4
Id:
Custom message:
Severity:
warning
Properties:
ignorePattern:
max:
80
tabWidth:
4
Translate tokens
Sort tokens
Default
Disable all but Line Length
- Edit and change tab width to 4


Whitespace
• Enable and disable the items selected below
Checkstyle Configuration
Internal Configuration "CS 170 Coding Guidelines"
Edit checkstyle configuration.
Disabled
eclipse-cs
The Checkstyle Plug-in for Eclipse
Known modules
Input filter text here
Size Violations
Whitespace
Empty For Initializer Pa
Empty For Iterator Pad
No Whitespace After
No Whitespace Before
Operator Wrap
Method Parameter Pad
Paren Pad
Tab Character
Typecast Paren Pad
Configured modules for group "Whitespace"
Enabled
Module
Severity
Comment
Empty For Iterator Pad ignore
Method Parameter Pad ignore
Operator Wrap ignore
Paren Pad ignore
Typecast Paren Pad ignore
No Whitespace After warning
No Whitespace Before warning
Tab Character warning
Whitespace After warning
Whitespace Around warning
Enabled

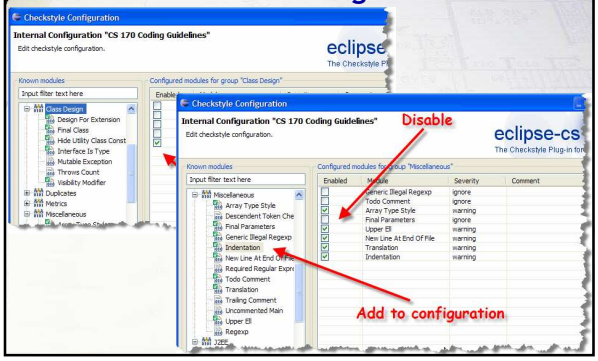

Modifiers and Blocks
• Ignore all modifiers, change brace position on blocks
Checkstyle Configuration
Internal Configuration "CS 170 Coding Guidelines"
Edit module configuration
Configuration of checkstyle module "Left Curly Brace Placem
Edit the module configuration.
Comment:
Braces changed to next line
Id:
Custom message:
Severity:
warning
Properties:
option:
nl
maxLineLength:
80
tokens:
Class declaration
Constructor declaration
Interface declaration
catch keyword
do keyword
else keyword
finally keyword
for keyword
if keyword
switch keyword
synchronized keyword
Translate tokens
Sort tokens
Known modules
Input filter text here
Imports
Size Violations
Whitespace
Modifiers
Blocks
Avoid Nested Blocks
Empty Block
Left Curly Brace Placement
Need Braces
Right Curly Brace Place
Coding Problems
Configured modules for group "Blocks"
Enabled
Module
Avoid Nested Blocks
Empty Block
Left Curly Brace Placement
Need Braces
Right Curly Brace Place...
Double-click

Coding Problems

- Many options. Let's add a check for switch fall-through
  - Note that this allows special comments when fall-through is desired
- May want to de-select the Magic Number and a few other checks

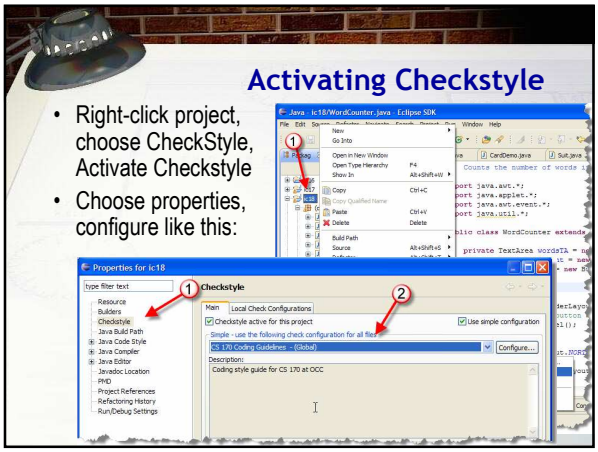


Class Design and Others



Activating Checkstyle

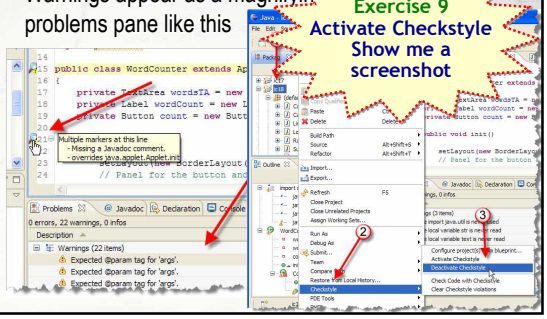
- Right-click project, choose CheckStyle, Activate Checkstyle
- Choose properties, configure like this:



Warnings and Disabling

- Warnings appear as a magnifyin problems pane like this

Exercise 9
Activate Checkstyle
Show me a screenshot



Finish Up

- In-class exercise: submit **before** you leave today!
  - Drop into Q:\faculty5\sgilbert\cs170\submissions
- Midterm Exam #2 (Chapters 5 and 6) tomorrow
- Reading Chapter 8 (not Chapter 7) for Thursday
  - Lightly skim 8.1, 8.2, 8.3, 8.4. 8.5, Skip 8.9
  - Study 8.6, 8.7, 8.8, 8.10